Überblick Zwischencode

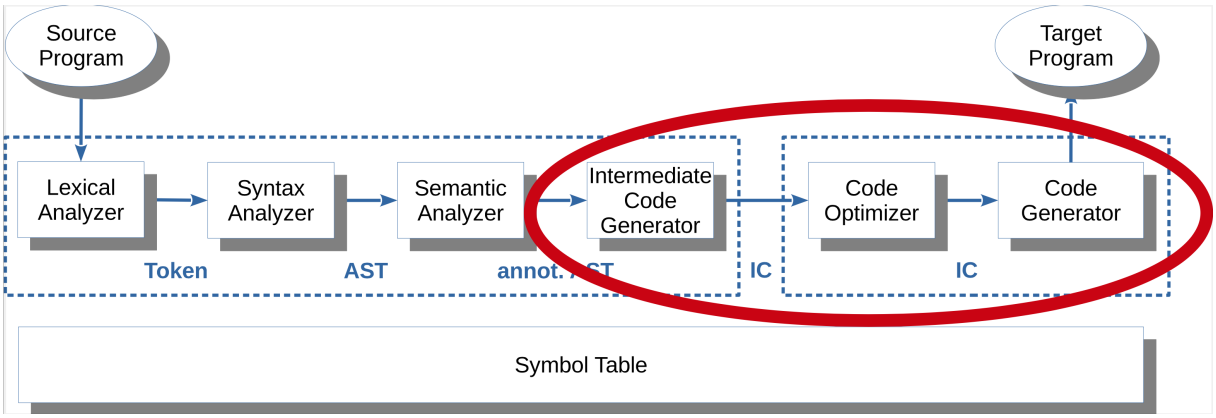
Carsten Gips (HSBI)

TL;DR
Für den Zwischencode (**IR**) gibt es kein allgemein definiertes Format. In der Praxis trifft man auf eine große Bandbreite an Formaten, wobei teilweise bereits der AST selbst als "Zwischencode" betrachtet/benutzt wird. Typische Vertreter für IR sind beispielsweise der **LLVM IR**, diverse Arten von Bytecode (nebst passender virtueller Maschine) und schließlich als Vorstufe für die Erzeugung von Maschinencode der sogenannte "Drei-Adressen-Code" und Assemblercode.
Videos

- VL Überblick Zwischencode

---

1 Einordnung



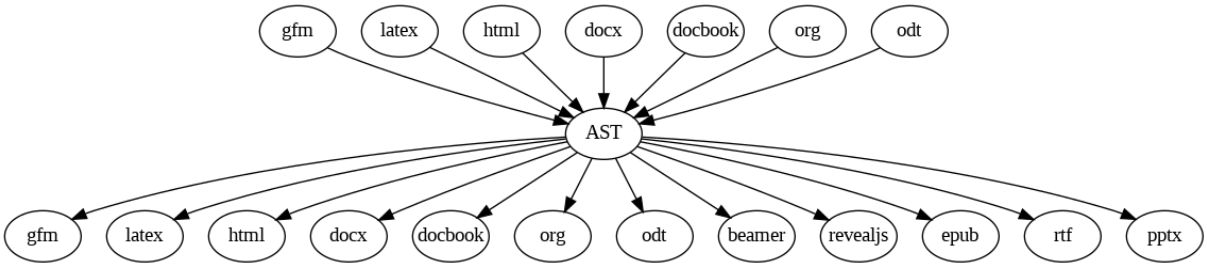
Die Schritte in der letzten Phase der Compiler-Pipeline können *sehr* unterschiedlich ausfallen.

Beispielsweise könnte direkt aus dem AST der Ziel-Machine-Code erzeugt werden. Auf der anderen Seite könnte aus dem AST ein **Zwischenformat** erzeugt werden, darauf Optimierungen vorgenommen werden, daraus ein weiteres Zwischenformat erzeugt werden, darauf weitere Optimierungen vorgenommen werden, …, bis schließlich nach mehreren Zwischenstufen das Zielformat erzeugt wird.

Nachfolgend betrachten wir verschiedene Beispiele, wie das Zwischenformat aussehen kann.

2 AST als Zwischencode (Beispiel Pandoc)

Häufig wird der AST selbst als Zwischencode verwendet. Ein Beispiel dafür ist Pandoc.




```
Dies ist ein Absatz mit
*  einem Stichpunkt, und
*  einem zweiten Stichpunkt.
```


```
{"blocks":[{"t":"Para","c":[{"t":"Str","c":"Dies"},
                            {"t":"Space"},
                            {"t":"Str","c":"ist"},
                            {"t":"Space"},
                            {"t":"Str","c":"ein"},
                            {"t":"Space"},
                            {"t":"Str","c":"Absatz"},
                            {"t":"Space"},
                            {"t":"Str","c":"mit"}]},
           {"t":"BulletList","c":[
               [{"t":"Plain","c":[{"t":"Str","c":"einem"},
                                  {"t":"Space"},
                                  {"t":"Str","c":"Stichpunkt,"},
                                  {"t":"Space"},
                                  {"t":"Str","c":"und"}]}],
               [{"t":"Plain","c":[{"t":"Str","c":"einem"},
                                  {"t":"Space"},
                                  {"t":"Str","c":"zweiten"},
                                  {"t":"Space"},
                                  {"t":"Str","c":"Stichpunkt."}]}]]}],
"pandoc-api-version":[1,17,0,4],"meta":{}}
```


Der Pandoc-AST spiegelt direkt die Dokumentstruktur wider. Im obigen Beispiel haben wir einen Absatz mit dem Text "`Dies ist ein Absatz mit`", der als `Para` repräsentiert wird mit einer Liste von Strings (`Str`) und Leerzeichen (`Space`).

Die Stichpunktliste besteht pro Stichpunkt aus einem `Plain`-Knoten mit dem eigentlichen Inhalt (wieder Strings und Leerzeichen).

Dieser AST ist der Dreh- und Angelpunkt in Pandoc. Verschiedene *Reader* können unterschiedliche Textformate parsen und in einen AST überführen.

Auf diesem kann man mit Filtern Transformationen vornehmen.

Anschließend können diverse *Writer* den AST in das gewünschte Zielformat überführen.

Konsole: pandoc hello.md -s -t native

3 Zwischenformat: Drei-Adressen-Code

Eine weitere häufig eingesetzte Zwischenform kurz vor der Code-Generierung ist der sogenannte "Drei-Adressen-Code". Dieser besteht jeweils aus einer Operation auf bis zu drei Adressen.

Im Prinzip handelt es sich hier um eine Art "High-Level Assembler" mit beliebig vielen Registern …

Adressen sind dabei Namen, Konstanten oder vom Compiler generierte temporäre Werte. Die typische Form ist `x = y op z` (binäre Operationen) oder `x = op z` (unäre Operationen). Werte werden mit `x = y` kopiert. Jeder Teilausdruck erhält typischerweise eine eigene temporäre Variable zur Speicherung des Ergebnisses. Weiterhin gibt es bedingte und unbedingte Sprünge und Prozedur-Aufrufe.

Index-Zugriffe werden über Pointerarithmetik aufgelöst (s.u.).

Eine Spezialform ist die sogenannte "Static Single-Assignment"-Form (*SSA*). Hierbei wird für jede Zuweisung eine neue temporäre Variable generiert, d.h. jede im IR-Code verwendete Adresse (temporäre Variable) hat genau eine Zuweisung. Dies wirkt sich günstig auf spezielle Optimierungen aus.

```
i = i+1;
if (a[i] >= v) {
    i = 0;
}
```

```
    t1 = i + 1
    i  = t1
    t2 = i * 8
    t3 = a + t2
    if t3 >= v goto L1
    goto L2
L1: i  = 0
L2: ...
```

Im obigen Beispiel wurde davon ausgegangen, dass die Einträge im Array `a` 8 Bit breit sind. Das muss der Compiler wissen, um jeweils den korrekten Offset zu benutzen.

Außerdem könnte man den Code gleich noch optimieren und die Anzahl der Sprünge reduzieren:

```
    t1 = i + 1
    i  = t1
    t2 = i * 8
    t3 = a + t2
    if t3 < v goto L
    i  = 0
L:  ...
```

4 LLVM IR

Low **L**evel **V**irtual **M**achine

```
int main() {
```

```
    int x = 7;
    int y = x + 35;

    return 0;
}
```

```
define i32 @main() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  store i32 0, i32* %1, align 4
  store i32 7, i32* %2, align 4
  %4 = load i32, i32* %2, align 4
  %5 = add nsw i32 %4, 35
  store i32 %5, i32* %3, align 4
  ret i32 0
}
```

Beispiel: clang -emit-llvm -S -o - hello.c

Der obige Output ist auf die *relevanten Zeilen gekürzt*; der gesamte Output im LLVM-Format sieht wie folgt aus:

```
; ModuleID = 'hello.c'
source_filename = "hello.c"
target datalayout = "e-m:e-i64:64-f80:128-n8:16:32:64-S128"
target triple = "x86_64-pc-linux-gnu"

; Function Attrs: noinline nounwind optnone uwtable
define i32 @main() #0 {
  %1 = alloca i32, align 4
  %2 = alloca i32, align 4
  %3 = alloca i32, align 4
  store i32 0, i32* %1, align 4
  store i32 7, i32* %2, align 4
  %4 = load i32, i32* %2, align 4
  %5 = add nsw i32 %4, 35
  store i32 %5, i32* %3, align 4
  ret i32 0
}

attributes #0 = { noinline nounwind optnone uwtable "correctly-rounded-divide-
    sqrt-fp-math"="false" "disable-tail-calls"="false" "less-precise-fpmad"="
    false" "no-frame-pointer-elim"="true" "no-frame-pointer-elim-non-leaf" "no-
    infs-fp-math"="false" "no-jump-tables"="false" "no-nans-fp-math"="false" "no
    -signed-zeros-fp-math"="false" "no-trapping-math"="false" "stack-protector-
    buffer-size"="8" "target-cpu"="x86-64" "target-features"="+fxsr,+mmx,+sse,+
    sse2,+x87" "unsafe-fp-math"="false" "use-soft-float"="false" }

!llvm.module.flags = !{!0}
!llvm.ident = !{!1}

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{!"clang version 6.0.0-1ubuntu2 (tags/RELEASE_600/final)"}
```

Es werden drei “virtuelle Register” (Variablen) `%1`, `%2` und `%3` auf dem Stack angelegt (32-bit Integer; `align 4`: alle Adressen sind Vielfache von 4).

Mit `store i32 0, ...` wird in %1 der Wert 0 geschrieben (vergleichbar mit `*p = 0`). In %2 wird analog der Wert 7 geschrieben (`x=7`).

Dann wird der Wert aus %2 in eine neue Variable %4 geladen und das Ergebnis der Addition aus %4 und dem Wert 35 in eine weitere neue Variable %5 geschrieben. Der Wert dieser Variablen wird dann auf dem Stack in %3 gespeichert (`y = x+35`).

Vgl. auch LLVM Language Reference Manual und blog.regehr.org/archives/1453.

5 Bytecode (Beispiel Python)

```
x = 7
y = x + 35
```

```
  1   0 LOAD_CONST     0 (7)
      3 STORE_NAME     0 (x)

  2   6 LOAD_NAME      0 (x)
      9 LOAD_CONST     1 (35)
     12 BINARY_ADD
     13 STORE_NAME     1 (y)
     16 LOAD_CONST     2 (None)
     19 RETURN_VALUE
```

Beispiel: python -m dis hello.py

Python pflegt 3 Listen: `co_names` für die Namen plus `co_values` für die dazugehörigen Werte sowie `co_consts` für Konstanten. Die Listen der Namen und Werte sind gleich lang, ein Index bezieht sich jeweils auf das selbe Symbol. Werte werden über einen Stack verarbeitet. Die Opcodes stehen in einer weiteren Liste `co_code`. (Die Opcodes sind oben der besseren Lesbarkeit halber als Text ausgegeben, `LOAD_CONST` hat beispielsweise den Wert `100`.)

Nach dem Laden des Programms ist `x` in `co_names[0]`, `y` in `co_names[1]`. Der Wert 7 steht in `co_const[0]`, die 35 in `co_const[1]`.

Das `LOAD_CONST 0` (`co_code[0]`) lädt den Inhalt von `co_consts[0]` auf den Stack (`push()`), d.h. der Wert 7 wird auf den Stack gepusht. Mit `STORE_NAME 0` (`co_code[3]`) wird der Inhalt des obersten Stackeintrags in `co_values[0]` geschrieben und der Eintrag vom Stack entfernt (`pop()`). Dies entspricht Zeile 1 im Quellcode: `x = 7`.

`LOAD_NAME 0` pusht `co_values[0]` auf den Stack (Wert von `x`), gefolgt von der 35 per `LOAD_CONST 1` (`co_const[1]`). Das `BINARY_ADD` entfernt die beiden obersten Einträge, addiert die Werte und pusht das Ergebnis wieder auf den Stack. Mit `STORE_NAME 1` wird der Wert in `co_values[1]` geschrieben, d.h. `y` bekommt den Wert zugewiesen.

6 Bytecode (Beispiel Java)

```java
public class Hello {

    void wuppie() {
        int x = 7;
        int y = x + 35;
   }

}
```

```
Compiled from "Hello.java"
public class Hello {
  public Hello();
    Code:
       0: aload_0
       1: invokespecial #1 // Method java/lang/Object."<init>":()V
       4: return

  void wuppie();
    Code:
       0: bipush        7
       2: istore_1
       3: iload_1
       4: bipush        35
       6: iadd
       7: istore_2
       8: return
}
```

Beispiel: javac Hello.java && javap -c Hello.class

Für jeden Methodenaufruf wird ein entsprechender Frame auf den Stack gepusht. Dieser enthält ein Array mit den lokalen Variablen, durchnummeriert von `0` bis `n-1`. (**`long`** und **`double`** bekommen je *2* lokale Variablen) Zusätzlich gibt es im Frame einen Operandenstack, auf dem Funktionsparameter und -rückgabewerte übergeben werden und auf dem die Operanden für die auszuführenden Operationen sowie deren Zwischenergebnisse hinterlegt werden.

- `bipush 7`: Pushe den Integer-Wert 7 auf den Stack
- `istore_1`: Poppe den ersten Wert vom Stack und speichere ihn in der lokalen Integer-Variable mit Index 1 (`x=7`)
- `iload_1`: Pushe lokale Integer-Variable mit Index 1 auf den Stack (`x`)
- `bipush 35`: Pushe den Integer-Wert 35 auf den Stack
- `iadd`: Führe Integer-Addition aus mit den beiden obersten Werten auf Stack und ersetze diese mit dem Ergebnis
- `istore_2`: Poppe den ersten Wert vom Stack und speichere ihn in der lokalen Integer-Variable mit Index 2 (`y=x+35`)

Die Konstanten `n` für `iconst_` funktionieren nur für kleinere Integer. Größere Werte muss man mit `bipush` auf den Stack pushen.

Vgl. auch dzone.com/articles/introduction-to-java-bytecode und www.beyondjava.net/java-programmers-guide-java-byte-code.

7 Assembler

```c
int main() {
    int x = 7;
    int y = x + 35;

    return 0;
}
```

```
    .file   "hello.c"
    .text
    .globl  main
    .type   main, @function
main:
.LFB0:
    .cfi_startproc
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    movl    $7, -8(%rbp)
    movl    -8(%rbp), %eax
    addl    $35, %eax
    movl    %eax, -4(%rbp)
    movl    $0, %eax
    popq    %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
.LFE0:
    .size       main, .-main
    .ident      "GCC: (Ubuntu 7.3.0-27ubuntu1~18.04) 7.3.0"
    .section    .note.GNU-stack,"",@progbits
```

Beispiel: gcc -S -o - hello.c

Die Ausgabe unterscheidet sich je nach Architektur, auf dem der C-Code in Assembler-Code compiliert wird!

Mit `.text` beginnt das Textsegment. `main:` ist eine Sprungmarke, die hier auch als Startpunkt für das Programm dient.

Auf X86-64 stehen `%rbp` und `%rsp` für 8-Byte-Register. Mit `%eax` greift man auf die Bytes 0 bis 3 des 8-Byte-Registers `%rax` zu.

Da in `%rbp` Werte übergeben werden (können), wird das Register mit `pushq %rbp` auf den Stack gesichert und am Ende mit `popq %rbp` wiederhergestellt.

Ansonsten kann man die Bedeutung erraten: `movl $7, -8(%rbp)` entspricht `mem[rbp-8] = 7`, `movl -8(%rbp), %eax` entspricht `eax = mem[rbp-8]`, `addl $35, %eax` entspricht `eax = eax + 35`, `movl %eax, -4(%rbp)` entspricht `mem[rbp-4] = eax`.

Vgl. auch cs.brown.edu/courses/cs033/docs/guides/x64_cheatsheet.pdf und en.wikibooks.org/wiki/X86_Assembly/GAS_Syntax.

8 Wrap-Up

- Compiler generieren aus AST Zwischencode (“*IC*” oder “*IR*”)
- Kein allgemein definiertes Format, große Bandbreite:
  - AST als IR
  - LLVM IR
  - Drei-Adressen-Code
  - Diverse Arten von Bytecode
  - Assemblercode

9 Zum Nachlesen

- Aho u. a. (2023): Kapitel 6 und 8
- Mogensen (2017): Kapitel 6 und 7
- Grune u. a. (2012): Kapitel 7
- Parr (2010): Kapitel 11

Lernziele

- k1: Varianten von Zwischencode, Vor- und Nachteile

Quellen

Aho, A. V., M. S. Lam, R. Sethi, J. D. Ullman, und S. Bansal. 2023. *Compilers: Principles, Techniques, and Tools, Updated 2nd Edition by Pearson*. Pearson India. https://learning.oreilly.com/library/view/compilers-principles-techniques/9789357054881/.

Grune, D., K. van Reeuwijk, H. E. Bal, C. J. H. Jacobs, und K. Langendoen. 2012. *Modern Compiler Design*. Springer.

Mogensen, T. 2017. *Introduction to Compiler Design*. Springer. https://doi.org/10.1007/978-3-319-66966-3.

Parr, T. 2010. *Language Implementation Patterns*. Pragmatic Bookshelf. https://learning.oreilly.com/library/view/language-implementation-patterns/9781680500097/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** a10718a (Remove Hugo: tooling: shift headings (use h1 as top-level headings), 2025-07-25)